

Entregable 3. Instructivo para levantar el entorno desde cero

Integrantes: Derly Dayana Garcia Carrillo · Orlando Rojas Narvaez | Docente: Juan Carlos Polania Cortes | Periodo 2026-2
| Repositorio: github.com/OrlandoRojasNa/taller02-rabbitmq-docker

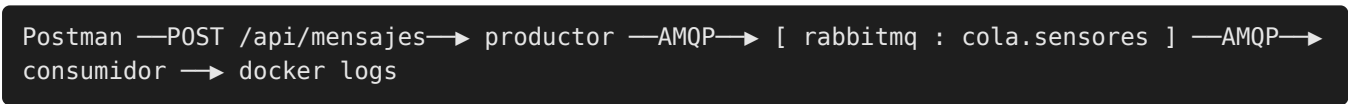
Asignatura: Sistemas Distribuidos · **Programa:** Ingeniería de Software · **Periodo:** 2026-2

Docente: Juan Carlos Polania Cortes

Integrantes: Derly Dayana Garcia Carrillo · Orlando Rojas Narvaez

Sistema de mensajería asíncrona **100 % dockerizado:**

Servicio	Qué hace	Puerto en el equipo
rabbitmq	Intermediario de mensajes, con la cola durable cola.sensores ya creada	15672 (consola)
productor	API REST (Java 21 + Spring Boot) que recibe JSON y lo publica en la cola	8080 (API)
consumidor	Servicio (Java 21 + Spring Boot) que lee la cola y muestra cada mensaje en sus logs	(ninguno)



1. Requisitos

Solo **Docker** (Docker Desktop en Windows/Mac, o Docker Engine + plugin Compose en Linux).
No hay que instalar Java, Maven ni RabbitMQ: todo se compila y se ejecuta dentro de las imágenes.

2. Levantar todo el entorno (un solo comando)

```
git clone https://github.com/OrlandoRojasNa/taller02-rabbitmq-docker.git
cd taller02-rabbitmq-docker
docker compose up -d --build
```

La primera vez tarda unos minutos porque descarga imágenes y dependencias Maven.
Para comprobar que los tres contenedores quedaron arriba:

```
docker compose ps
```

Recurso	URL / dato
Consola de RabbitMQ	http://localhost:15672 · usuario lab · contraseña lab123
Endpoint del productor	POST http://localhost:8080/api/mensajes
Cola	cola.sensores (durable, vhost /)
Logs del consumidor	docker compose logs -f consumidor

Para apagar: `docker compose down` (conserva los mensajes en el volumen).

Para borrar todo, incluidos los datos: `docker compose down -v`.

3. Cómo cumple cada requisito

Requisito del taller	Implementación			
Todo dockerizado	Dockerfiles multi-etapa: la etapa <code>maven:3.9-eclipse-temurin-21</code> compila y la etapa <code>eclipse-temurin:21-jre</code> ejecuta.			
Consola accesible	Imagen <code>rabbitmq:4.1-management</code> , puerto <code>15672</code> publicado.			
Cola durable que existe al levantar el entorno	<code>rabbitmq/definitions.json</code> se carga al arrancar el nodo (<code>load_definitions</code>). Además, productor y consumidor la declaran con los mismos atributos, lo cual es idempotente.			
Productor POST + JSON	<code>POST /api/mensajes</code> . Valida que el cuerpo sea un objeto JSON y publica con <code>deliveryMode = PERSISTENT</code> . Espera la confirmación del broker (<i><code>publisher confirms</code></i>) antes de responder <code>"Mensaje encolado"</code> .			
<code>status</code> booleano	Se responde con <code>record Respuesta(boolean status, String message)</code> , así que Jackson lo serializa como <code>true / false</code> , no como texto.			
Consumidor muestra contenido y hora	Log: <code>`[#n] Mensaje recibido \</code>	hora de recepcion: ... \	id: ... \	contenido: {...} (zona horaria America/Bogota`).
Retirar el mensaje solo después de procesarlo	<code>acknowledge-mode: manual + basicAck</code> al terminar de procesar. Si falla, <code>basicNack</code> con reencolado. <code>prefetch: 1</code> .			
Un único archivo de composición	<code>docker-compose.yml</code> .			
Servicios por nombre, no por <code>localhost</code>	Las apps se conectan a <code>SPRING_RABBITMQ_HOST=rabbitmq</code> . El puerto <code>AMQP 5672</code> no se publica al equipo.			
Reintentar si RabbitMQ no está listo	<code>depends_on: condition: service_healthy</code> + reintentos propios de Spring AMQP (el listener reintentará cada 5 s y el productor se conecta en cada publicación).			
Persistencia ante reinicios	Cola durable + mensajes persistentes + volumen <code>rabbitmq_data</code> + <code>hostname: rabbitmq</code> fijo (el nombre del nodo no cambia).			

Respuestas de la API

Caso	HTTP	Cuerpo
Mensaje publicado	200	<code>{"status": true, "message": "Mensaje encolado"}</code>
Cuerpo vacío, no JSON, JSON mal formado o que no es un objeto	400	<code>{"status": false, "message": "Formato de mensaje inválido"}</code>
RabbitMQ no disponible	503	<code>{"status": false, "message": "No se pudo publicar el mensaje en la cola"}</code>

Todas las respuestas llevan `Content-Type: application/json`.

4. Pruebas con Postman

- Importe `postman/Taller02-RabbitMQ.postman_collection.json`.
- Peticiones incluidas:
- **Enviar mensaje válido:** incluye tests que verifican `status === true` (boolean).
- **Enviar mensaje inválido (JSON mal formado) y (texto plano):** verifican `status === false`.
- **Estado de la cola:** consulta la API de administración de RabbitMQ (mensajes listos, sin ACK, consumidores).
- **10 mensajes válidos:** *Runner* → marque solo *Enviar mensaje válido* → *Iterations = 10* → *Run*.

Luego verifique en la consola (*Queues and Streams* → *cola.sensores*) y en `docker compose logs consumidor` que llegaron los 10.

5. Casos a comprobar (punto 7 del taller)

Tome capturas **antes y después** en la consola: *Queues and Streams* → *cola.sensores* (columnas *Ready*, *Unacked*, *Total* y la gráfica de *Queued messages*).

Guárdelas en `docs/evidencias/`. Allí está la lista de capturas, cuáles faltan y el paso a paso para tomarlas.

Caso 1: consumidor detenido

```
docker compose stop consumidor
```

Envíe varios mensajes válidos desde Postman. En la consola, *Ready* aumenta y *Consumers* = 0.

```
docker compose start consumidor
docker compose logs -f consumidor
```

El consumidor procesa todos los mensajes acumulados y *Ready* vuelve a 0.

Caso 2: reinicio de RabbitMQ con mensajes pendientes

```
docker compose stop consumidor
```

Envíe varios mensajes y anote la cantidad en *Ready*.

```
docker compose restart rabbitmq
```

Espere unos segundos, recargue la consola (vuelva a iniciar sesión) y verifique que la cola tiene **la misma cantidad** de mensajes. Luego:

```
docker compose start consumidor
```

Para una prueba aún más fuerte, puede recrear el contenedor: `docker compose rm -sf rabbitmq && docker compose up -d rabbitmq`. Los mensajes siguen ahí gracias al volumen.

Caso 3: mensaje inválido

Anote el total de la cola, envíe *Enviar mensaje invalido* desde Postman y verifique:

- la respuesta es `{"status": false, "message": "Formato de mensaje inválido"};`
- el total de la cola **no cambia**;
- en `docker compose logs productor` aparece `Mensaje rechazado, no se publica: ...`.

6. Comandos útiles

```
docker compose logs -f productor          # logs del productor
docker compose logs -f consumidor         # mensajes recibidos
docker compose exec rabbitmq rabbitmqctl list_queues name durable messages_ready
messages_unacknowledged consumers
```

Ejemplo con `curl`:

```
curl -i -X POST http://localhost:8080/api/mensajes -H "Content-Type: application/json" -d '{"sensor": "OD-01", "valor": 4.2, "unidad": "mg/L"}'
```

7. Estructura del proyecto

```
taller02-rabbitmq-docker/
├─ docker-compose.yml          # levanta los 3 servicios
├─ rabbitmq/                   # imagen del broker + definiciones (cola durable)
├─ productor/                  # API Spring Boot (Dockerfile + código + tests)
├─ consumidor/                 # listener Spring Boot (Dockerfile + código)
├─ postman/                    # colección exportada
├─ docs/
│   └─ DECLARACION_IA.md       # declaración de uso de IA (obligatoria)
│   └─ evidencias/             # capturas de pantalla
└─ pom.xml                     # agregador, solo para abrir todo en IntelliJ
```

8. Abrir en IntelliJ IDEA

File → *Open* → seleccione la carpeta raíz (el `pom.xml` agregador carga `productor` y `consumidor` como módulos).

IntelliJ se usa solo como **editor**. La ejecución es siempre con `docker compose up -d --build`.

9. Declaración de uso de IA

Ver `docs/DECLARACION_IA.md` .